



# Reproducibility Report for MeshAgent

Analysis of the Implemented Work and Reproducibility Assessment

Jorge Echevarria de Uribarri

EURECOM

5 June 2026

Logo source: EURECOM newsroom [3].

# Contents

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                              | <b>1</b>  |
| <b>2</b>  | <b>Methodological Framework</b>                  | <b>1</b>  |
| <b>3</b>  | <b>Baseline Repository State</b>                 | <b>2</b>  |
| <b>4</b>  | <b>Implemented Changes</b>                       | <b>3</b>  |
| <b>5</b>  | <b>Current Architecture and Toolchain</b>        | <b>4</b>  |
| <b>6</b>  | <b>Experimental Setup and Available Evidence</b> | <b>5</b>  |
| <b>7</b>  | <b>Reproducibility Assessment</b>                | <b>6</b>  |
| <b>8</b>  | <b>Pending Comparative Model Evaluation</b>      | <b>7</b>  |
| <b>9</b>  | <b>Discussion</b>                                | <b>8</b>  |
| <b>10</b> | <b>Conclusions and Recommendations</b>           | <b>8</b>  |
| <b>A</b>  | <b>Detailed Evidence Map</b>                     | <b>10</b> |
| <b>B</b>  | <b>Commit Evidence</b>                           | <b>11</b> |
| <b>C</b>  | <b>Reproduction Commands</b>                     | <b>11</b> |
| <b>D</b>  | <b>Environment Variables</b>                     | <b>12</b> |
| <b>E</b>  | <b>Pending Items Checklist</b>                   | <b>13</b> |

## Abstract

This report evaluates the reproducibility of the current MeshAgent repository as a technical artifact and as an empirical basis for LLM-assisted network-graph analysis. The repository has been migrated from an Azure-dependent implementation to a local, self-hostable stack based on vLLM, `sentence-transformers`, and Qdrant. That migration makes the system more inspectable and substantially easier to run than the upstream baseline, while preserving the original agentic workflow of retrieval, LLM code generation, execution, verification, and golden-answer comparison.

Following the reproducibility framework used in the provided NetLLM thesis [7] and the ACM artifact criteria [1], the report concludes that MeshAgent is *partially reproducible*. The code, graph data, prompt templates, RAG sources, persisted Qdrant collections, smoke-test scripts, and result logs are available. However, the exact model-serving environment is not fully specified, most dependencies are not pinned, vLLM is installed outside the repository dependency file, and the comparative model evaluation has not yet been executed. Consequently, the implementation is technically credible, but its empirical reproducibility remains incomplete.

## 1 Introduction

LLM-based systems complicate the ordinary meaning of a reproducible software artifact. Their behavior is shaped not only by source code and data, but also by model checkpoints, inference backends, prompt templates, vector indexes, hardware, and package versions. A small change in any of these layers can alter the generated output, especially when the LLM is part of the runtime system rather than merely a tool used during development.

The provided thesis on NetLLM frames this problem in the context of network research, where reproducing results requires attention to code availability, model availability, hardware constraints, dependency versions, seeds, and evaluation scripts [7, 9]. MeshAgent belongs to the same broad family of systems: it uses an LLM to generate executable Python code for network and graph-analysis tasks, combines that generation with retrieval over constraints and tools, and evaluates outputs against verifiers and golden answers when possible.

The purpose of this report is therefore twofold. First, it documents the engineering work implemented in the repository, especially the migration away from cloud-specific Azure services toward a local stack. Second, it assesses how far the resulting artifact can support reproduction by an external evaluator. The answer is deliberately cautious: the repository is much more reproducible than the upstream baseline, but it is not yet a complete reproducibility package.

## 2 Methodological Framework

### 2.1 Terminology

This report follows the distinction used in the thesis and in the ACM artifact-review criteria [7, 1]. *Reproduction* refers to obtaining the same results using the same artifacts and experimental setup. *Replication* refers to obtaining consistent results with a different or independently recreated setup. In this report, most claims concern reproduction of the current repository state. Claims about replication are limited, because the planned cross-model comparison has not yet been performed.

LLM-based artifacts present specific risks: dependency drift, unavailable model checkpoints, hidden prompts, non-deterministic inference, undocumented hardware requirements, unpinned vector-store state, and incomplete evaluation scripts. These risks are not abstract in MeshA-

gent. They appear concretely in the need to start an external vLLM server, resolve Hugging Face-hosted models, rebuild or trust Qdrant collections, and interpret small result logs that are not statistically controlled benchmarks.

## 2.2 Method of Assessment

The assessment was produced through a repository audit rather than by relying on the repository documentation alone. The following evidence was inspected:

- git history comparing `upstream/main` with the current `main` branch;
- README files, environment examples, dependency files, and benchmark documentation;
- the shared `common/` modules for configuration, LLM access, embeddings, vector search, RAG indexing, and graph compatibility;
- the three application entrypoints and helper modules;
- graph data, RAG JSON files, golden-answer files, persisted Qdrant metadata, and included result logs;
- smoke-test outcomes for embeddings, retrieval, and vLLM connectivity;
- the provided thesis PDF as the methodological reference.

### Evidence note

The current local branch is `main` at `84218ab`. The upstream baseline used for comparison is `0fb45c7`. Full hashes are listed in appendix B; the narrative uses short hashes for readability.

## 3 Baseline Repository State

### 3.1 Reconstructing the Baseline

The baseline state can be reconstructed from git history rather than inferred from file names alone. The upstream branch points to `0fb45c7`, after an initial import and cleanup. The current branch diverges after that point, with later commits adding the local execution stack, Qdrant configuration, smoke tests, result logs, and benchmark wrapper.

The upstream repository implemented an agentic graph-analysis workflow across three application directories: `app-malt`, `app-CRG`, and `app-traffic-analysis`. The workflow already had the essential MeshAgent structure: retrieve task constraints and tools, ask an LLM to produce a plan, generate Python code for a `process_graph` function, execute that code on a graph, run verifier checks, self-debug failures, and compare outputs against golden answers when available.

### 3.2 Baseline Dependencies and Reproducibility Limits

The baseline was difficult to reproduce because its infrastructure depended on externally configured cloud services. The root README listed notebook-style package installation commands rather than a project-level dependency file. It also required Azure Search, Azure OpenAI or Google VertexAI configuration, and Jupyter notebooks for RAG index creation. The upstream

`app-malt/ai_models_cot.py` used Azure GPT deployments such as `gpt-4-32k`; helper code imported Azure Search clients and vector-search classes.

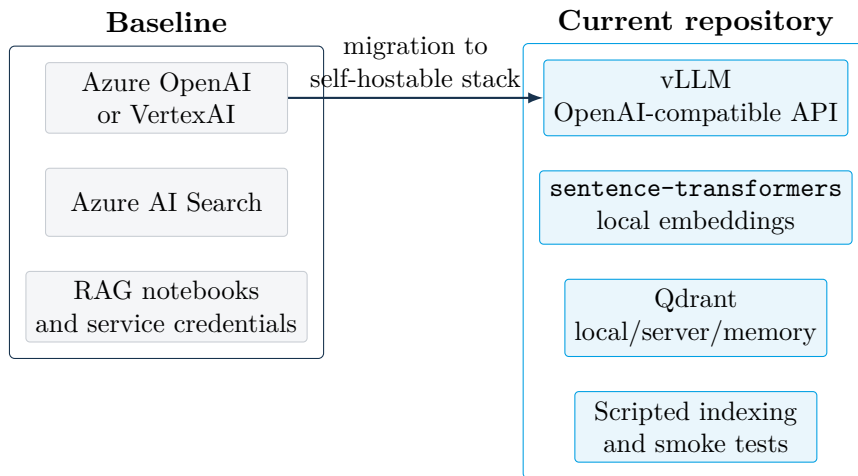
This baseline was not unusable, but it was not self-contained. An evaluator needed credentials, service endpoints, model deployment names, Azure index names, and notebook execution steps. In the terminology of the ACM criteria, the artifacts were available but not straightforwardly exercisable [1]. The problem resembles the NetLLM reproduction issues discussed in the thesis: code availability alone does not guarantee reproduction if the environment, dependencies, and models are underspecified [7].

## 4 Implemented Changes

### 4.1 Migration Logic

The current implementation follows a coherent engineering strategy. It replaces credential-dependent cloud services with local components while leaving the original agentic control flow mostly intact. Instead of rewriting the graph-analysis pipeline, the implementation changes the provider layer underneath it.

The resulting stack has three local components: `vLLM` provides chat inference, `Sentence-Transformers` computes embeddings, and `Qdrant` stores and searches RAG constraints and tools. Configuration moves into `.env.example` and `common/config.py`. Retrieval moves into `common/vector_store.py`, while index construction moves from Azure notebooks into `scripts/build_qdrant_indexes.py`. Figure 1 summarizes the architectural change.



**Figure 1:** Conceptual architecture change from the upstream cloud-dependent baseline to the current local/self-hostable implementation.

### 4.2 Commit Summary

The table deliberately compresses the commit history. The more important point is the migration path: the repository now has a provider layer that can be inspected and tested independently of the application prompts. This separation is the strongest engineering contribution from a reproducibility standpoint.

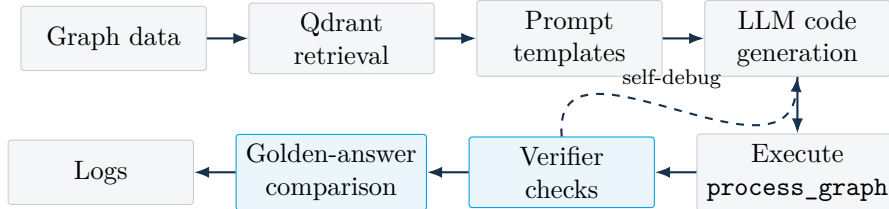
**Table 1:** Concise implementation history.

| Short commit      | Main change                    | Reproducibility impact                                                               |
|-------------------|--------------------------------|--------------------------------------------------------------------------------------|
| 7b099a2           | Local vLLM/Qdrant migration    | Removes the hard dependency on Azure services and introduces shared local providers. |
| 9283e88           | Flexible Qdrant modes          | Allows embedded local storage, server mode, or memory mode.                          |
| 993f3d6           | Log-directory creation         | Avoids failures caused by missing output directories.                                |
| 670e5c8           | NetworkX compatibility         | Handles node-link JSON differences between <b>links</b> and <b>edges</b> .           |
| 9a80e9b / db3b005 | Missing-golden-answer handling | Allows inference-only or skipped runs instead of aborting the whole evaluation.      |
| 6241057           | Result and Qdrant artifacts    | Provides inspectable logs and persisted local vector collections.                    |
| 84218ab           | Benchmark-result ignore rule   | Keeps generated benchmark outputs out of normal repository churn.                    |

## 5 Current Architecture and Toolchain

### 5.1 Execution Flow

The current pipeline remains an LLM-driven code-generation system. Graph data are loaded from checked-in files, RAG retrieval selects relevant constraints and tools, prompt templates ask the LLM for a three-step plan and step-specific code, and the generated `process_graph` function is executed. Verifier checks and self-debugging loops attempt to catch structural or execution failures before the final answer is compared with a golden answer.



**Figure 2:** Simplified MeshAgent execution flow. The dashed arrow represents the self-debugging loop triggered by execution or verifier errors.

### 5.2 Toolchain

The toolchain is now more transparent than in the upstream repository, but it remains a layered system. vLLM and the selected model define generation behavior [4, 8]; Sentence-Transformers and Qdrant define retrieval behavior [6, 5]; NetworkX and app-specific helpers define graph semantics; and the evaluation scripts define what is counted as a pass or failure. Table 2 summarizes the most relevant components.

This stack is appropriate for a reproducibility-oriented migration because it moves the system toward inspectable local services. At the same time, it does not by itself guarantee reproducibility. The model-serving command, model revision, embedding revision, and package versions remain part of the experimental artifact and should be recorded with every final run.

**Table 2:** Main technical components and their reproducibility role.

| Component                 | Role                                                       | Reproducibility significance                                                                                       |
|---------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| vLLM                      | Serves an OpenAI-compatible chat endpoint.                 | Makes local serving possible, but exact vLLM version, model revision, quantization, and hardware must be recorded. |
| Qdrant                    | Stores and searches embedded constraints and tools.        | Provides local vector-store state; rebuilds depend on embedding model and package versions.                        |
| Sentence-Transformers     | Computes local embeddings.                                 | Removes Azure embedding dependence; still depends on a Hugging Face-hosted model unless cached or pinned.          |
| LangChain / OpenAI client | Provides chat and prompt interfaces.                       | Preserves existing chain structure; API behavior can drift across versions.                                        |
| NetworkX                  | Represents graph inputs and generated graph outputs.       | Requires compatibility handling for node-link JSON formats.                                                        |
| Benchmark wrapper         | Runs smoke tests and app endpoints across model endpoints. | Enables the pending cross-model evaluation, but no final benchmark output is present.                              |

## 6 Experimental Setup and Available Evidence

### 6.1 Repository and Environment

The inspected repository is <https://github.com/Echeva26/MeshAgent.git> [2]. The current branch is `main`; the baseline comparison uses `upstream/main`. During the audit, the local virtual environment used Python 3.14.2 on macOS arm64. This local environment is not necessarily the environment used to produce the included result logs.

#### Missing information

The exact hardware, GPU model, vLLM version, CUDA version, PyTorch CUDA build, model revision, quantization mode, and model-serving command used for the included result logs are not recorded in the repository.

### 6.2 Data and Index Evidence

The graph datasets are checked into the repository. Loading them from the required app directories produced the sizes shown in table 3. The need to run from app directories is not a cosmetic detail: the helpers use relative paths.

The RAG sources are also versioned: MALT has 15 constraints and 4 tools, CRG has 15 constraints and 2 tools, and Traffic Analysis has 10 constraints and 5 tools. The persisted Qdrant metadata contains matching collection counts, all using 384-dimensional cosine vectors, which is consistent with the default `all-MiniLM-L6-v2` embedding model.

**Table 3:** Checked-in graph datasets.

| App              | Nodes | Edges | Source                                         |
|------------------|-------|-------|------------------------------------------------|
| MALT             | 5,493 | 6,424 | app-malt/data/malt-example-final.textproto.txt |
| CRG              | 200   | 1,034 | app-CRG/data/resources.json                    |
| Traffic Analysis | 100   | 32    | app-traffic-analysis/data/test_graph.json      |

### 6.3 Smoke Tests

The embedding and retrieval checks provide evidence that part of the local stack is functional. The embedding test initially required access to Hugging Face in order to resolve the configured model; after that access was available, it returned 384-dimensional vectors. The retrieval test against `app-malt` returned Qdrant constraint and tool matches. The vLLM test failed with connection refused, indicating that no OpenAI-compatible model server was listening at the configured endpoint during the audit.

These outcomes should be read carefully. They show that the local embedding and retrieval layers can run, not that the full application is currently reproducible end to end. Full execution still requires a separately installed and running vLLM server.

## 7 Reproducibility Assessment

### 7.1 Logged Results

The repository includes result logs for all three applications. They demonstrate that the pipeline has previously been run through code generation, execution, verification, and output comparison. They do not constitute a statistically controlled benchmark.

**Table 4:** Summary of included result logs.

| App              | Evaluated | Passed | Failed | Skipped |
|------------------|-----------|--------|--------|---------|
| MALT             | 3         | 1      | 2      | 1       |
| CRG              | 0         | 0      | 0      | 3       |
| Traffic Analysis | 6         | 3      | 3      | 1       |
| Total            | 9         | 4      | 5      | 5       |

Among prompts with golden answers, the included logs show a pass rate of  $4/9 = 44.4\%$ . This number is useful only as a descriptive summary of the available logs. It is not a stable estimate of model performance because the sample is small, run metadata are incomplete, and some failures may reflect strict golden-answer matching rather than an unequivocally wrong answer. Conversely, verifier success alone does not prove semantic correctness: several logged failures passed structural checks but missed the intended aggregation or grouping.

The skipped cases are also informative. They show that the current code can avoid aborting when an exact golden-answer key is missing, but they also show that some active prompts are not aligned with the available evaluation set. For academic evaluation, the prompt list, golden-answer file, and result summary should be synchronized before the final run.



## 7.2 Artifact Assessment

**Table 5:** Reproducibility assessment summary.

| Criterion                             | Status  | Rationale                                                                                                         |
|---------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------|
| Artifacts available                   | YES     | Source code, graph data, prompts, RAG inputs, persisted Qdrant storage, and logs are present.                     |
| Artifacts functional                  | PARTIAL | Embedding and retrieval smoke tests passed; vLLM connectivity failed because no server was running.               |
| Environment reproducibility           | PARTIAL | Configuration is documented, but no lockfile, Dockerfile, Python version constraint, or vLLM version is provided. |
| Dependency reproducibility            | PARTIAL | A <code>requirements.txt</code> exists, but most dependencies are unpinned and vLLM is installed separately.      |
| Model reproducibility                 | PARTIAL | Model names are specified, but exact revisions, serving settings, and quantization are not recorded.              |
| Data and vector-store reproducibility | PARTIAL | Data and Qdrant collections are present; regenerated embeddings may differ if model or package versions drift.    |
| Results reproducible                  | PARTIAL | Logs demonstrate prior execution, but a full rerun requires the missing serving environment.                      |
| Results replicable                    | PENDING | The comparative model experiment has not yet been executed.                                                       |

The overall assessment is therefore **partially reproducible**. The repository is no longer merely a cloud-dependent research prototype, but it still lacks the environmental precision required for independent reproduction of the included results from scratch.

## 8 Pending Comparative Model Evaluation

### Pending item

The final comparative evaluation across open-source model backends has not been executed. This section records the planned experiment and the evidence required to complete it. Leaving the results pending is methodologically necessary: reporting synthetic or assumed benchmark values would misrepresent the current repository evidence.

The benchmark wrapper in `scripts/benchmark_opensource_models.py` is designed to run the existing smoke tests and application entrypoints across several OpenAI-compatible model endpoints. The example matrix lists Qwen2.5-Coder, DeepSeek-Coder, and StarCoder2 endpoints. Once the servers are running, the wrapper should collect return codes, durations, query counts, accuracy values, pass/fail counts, skipped counts, and captured stdout/stderr.

For the final submission, the benchmark output should also record GPU model, VRAM, vLLM version, CUDA version, PyTorch version, model revision, quantization mode, maximum context length, and exact server command for each model. Without that metadata, cross-model claims would remain difficult to interpret.

**Table 6:** Planned comparative model results table.

| Model                        | Tests run | Successes | Mean accuracy | Duration | Notes                         |
|------------------------------|-----------|-----------|---------------|----------|-------------------------------|
| Qwen2.5-Coder-14B-Instruct   | Pending   | Pending   | Pending       | Pending  | Awaiting benchmark execution. |
| DeepSeek-Coder-6.7B-Instruct | Pending   | Pending   | Pending       | Pending  | Awaiting benchmark execution. |
| StarCoder2-15B-Instruct      | Pending   | Pending   | Pending       | Pending  | Awaiting benchmark execution. |

## 9 Discussion

The repository is strongest as an engineering migration. It converts a credential-heavy Azure workflow into a local architecture whose components can be inspected, tested, and replaced. The shared modules under `common/` are particularly valuable: they isolate configuration, model access, embeddings, Qdrant retrieval, and graph-format compatibility from the application-specific prompt logic.

The remaining weaknesses are typical of LLM-centered reproducibility work. A model name is not the same as a fixed model artifact; an OpenAI-compatible endpoint is not the same as a recorded inference environment; and temperature zero does not eliminate every source of backend- or hardware-dependent behavior. Similarly, persisted Qdrant storage is helpful, but regenerated vectors may differ if the embedding model revision or package version changes.

The result logs also reveal a distinction between operational execution and task correctness. Some generated code can execute and pass verifier checks while still answering the user query incorrectly. This is not simply a bug in the migration. It reflects a deeper limitation of the evaluation design: structural verifiers catch graph-format and invariant problems, whereas semantic correctness still depends on the golden-answer comparison and on whether the golden answer admits multiple valid outputs.

At EURECOM submission level, the project is credible but not empirically complete. It documents a meaningful reproducibility intervention and provides evidence of partial execution. Before final submission, the comparative model evaluation and environment metadata should be added so the report can move from an implementation audit to a complete empirical study.

## 10 Conclusions and Recommendations

MeshAgent should be classified as **partially reproducible**. The current repository is materially stronger than the upstream baseline: it replaces Azure-specific services with local/self-hostable infrastructure, provides documented configuration, adds scriptable RAG indexing, includes smoke tests, and preserves result logs and Qdrant state. These changes make the artifact easier to inspect and more plausible for external evaluation.

The empirical record is not complete. A full reproduction still requires a vLLM server and model-serving environment that are not fully specified in the repository. Dependencies are insufficiently pinned, model revisions are not fixed, and the final model-comparison experiment has not been run. The implementation is therefore mature enough to analyze, but the

empirical evaluation is not yet mature enough to support claims about cross-model robustness or replication.

The most important final-submission actions are:

1. pin Python dependencies or provide a lockfile;
2. document the supported Python, CUDA, PyTorch, vLLM, and GPU environment;
3. record exact LLM and embedding model revisions;
4. provide a Dockerfile, Docker Compose file, or equivalent reproducible environment recipe;
5. attach environment metadata to every result log;
6. run and archive the comparative model benchmark;
7. include the final model-comparison table and figure only after execution.

## A Detailed Evidence Map

This appendix records the files used as repository evidence. The main text refers to these artifacts selectively; the table below keeps the detailed file map out of the narrative.

**Table 7:** Repository evidence inspected during the audit.

| Path                                                                                  | Evidence value                                                                           |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| README.md                                                                             | Local setup instructions for vLLM, Qdrant, RAG indexing, smoke tests, and app execution. |
| .env.example                                                                          | Environment template for LLM, embedding, and Qdrant settings.                            |
| requirements.txt                                                                      | Dependency list; most packages are unpinned.                                             |
| CHANGE_ANALYSIS_AND_RESULTS.md                                                        | Repository-authored change summary and interpretation of included logs.                  |
| BENCHMARKING_OPEN_SOURCE_MODELS.md                                                    | Usage guide for the model-comparison wrapper.                                            |
| common/config.py                                                                      | Central runtime configuration.                                                           |
| common/llm_provider.py                                                                | vLLM/OpenAI-compatible chat provider.                                                    |
| common/embeddings_provider.py                                                         | Local embedding provider.                                                                |
| common/vector_store.py                                                                | Qdrant client and search functions.                                                      |
| common/rag_indexer.py                                                                 | Scriptable local RAG index construction.                                                 |
| common/chain_compat.py                                                                | Compatibility wrapper for legacy chain calls.                                            |
| common/graph_json.py                                                                  | NetworkX node-link compatibility helper.                                                 |
| scripts/test_embeddings.py                                                            | Embedding smoke test.                                                                    |
| scripts/test_retrieval.py                                                             | Qdrant retrieval smoke test.                                                             |
| scripts/test_vllm.py                                                                  | vLLM endpoint smoke test.                                                                |
| scripts/benchmark_opensource_models.py                                                | Pending model-comparison runner.                                                         |
| scripts/benchmark_models.example.json                                                 | Example matrix for Qwen, DeepSeek-Coder, and StarCoder2.                                 |
| app-*/ai_models_cot.py                                                                | Prompt templates and LLM chain definitions.                                              |
| app-*/full_cot_with_tools.py                                                          | Main generation, execution, verification, and logging loops.                             |
| app-*/helper.py                                                                       | Graph loading and output cleanup logic.                                                  |
| app-*/error_check.py                                                                  | App-specific verifier logic.                                                             |
| app-*/data/rag_constraints.json                                                       | RAG constraint sources.                                                                  |
| app-*/data/rag_tools.json                                                             | RAG tool sources.                                                                        |
| app-*/golden_answer_generator/prompt_golden_ans.json                                  | Golden-answer files used for accuracy checks.                                            |
| qdrant_storage/                                                                       | Persisted local Qdrant storage.                                                          |
| app-malt/results.txt,<br>app-CRG/results.txt,<br>app-traffic-analysis/<br>results.txt | Included result logs.                                                                    |

## B Commit Evidence

Full hashes are included here for traceability while the main report uses short hashes.

**Table 8:** Commit identifiers used in the report.

| Short   | Full hash                                | Role                                          |
|---------|------------------------------------------|-----------------------------------------------|
| 84218ab | 84218abadccb2a5f855f5bef4dcbe07b53af996a | Current inspected commit.                     |
| db3b005 | db3b005e72b4ca60664b37b6c8825297ac7b72a4 | MALT missing-golden handling.                 |
| 6241057 | 6241057e0f5fd38f9664ac6c8cf47cf459ed0d84 | Result logs and Qdrant storage.               |
| 0a24c86 | 0a24c8699695b07d06786d598185efceb2e41c6c | Defensive CRG verifier handling.              |
| 9a80e9b | 9a80e9b7c9c208b06cc9776bc243d3fa755b5dcd | Inference-only missing-ground-truth handling. |
| 670e5c8 | 670e5c847a128310ef66fba59a24f59c30546cce | NetworkX JSON compatibility.                  |
| 993f3d6 | 993f3d6271c5307249702f53eb53256c7b719bcb | Log-directory creation.                       |
| 9283e88 | 9283e886a795f218daa8978394906590b513f5c3 | Flexible Qdrant modes.                        |
| 7b099a2 | 7b099a2d10eb212ba4a7b338a9d7cd99662865dd | Local vLLM/Qdrant migration.                  |
| 0fb45c7 | 0fb45c75ac5cd468ccd5946a0b99f7062a5e5d3e | Upstream baseline.                            |

## C Reproduction Commands

The following command groups document how the audit was performed and how a future evaluator can reproduce the checks. They are not a substitute for a full benchmark run.

### C.1 Repository Inspection

```
git status --short
git branch --all --verbose --no-abbrev
git log --oneline --decorate --graph --all --max-count=80
git diff --stat upstream/main...main
git diff --name-status upstream/main...main
```

### C.2 Baseline Comparison

```
git show upstream/main:README.md
git show upstream/main:app-malt/README.md
git show upstream/main:app-malt/ai_models_cot.py
git show upstream/main:app-malt/helper.py
```

### C.3 Smoke Tests

```
.venv/bin/python --version
.venv/bin/python -m pip freeze
.venv/bin/python scripts/test_embeddings.py
.venv/bin/python scripts/test_retrieval.py --app app-malt
.venv/bin/python scripts/test_vllm.py
```

Observed outcome: embeddings and retrieval passed after model access was available; vLLM connectivity failed because no server was listening at the configured endpoint.

### C.4 Graph-Size Checks

```
cd app-malt
../.venv/bin/python -c "from helper import getGraphData; raw,G=getGraphData();
    print(G.number_of_nodes(), G.number_of_edges())"

cd ../app-CRG
../.venv/bin/python -c "from helper import getGraphData; G=getGraphData();
    print(G.number_of_nodes(), G.number_of_edges())"

cd ../app-traffic-analysis
../.venv/bin/python -c "from helper import getGraphData; G=getGraphData();
    print(G.number_of_nodes(), G.number_of_edges())"
```

### C.5 Pending Benchmark Command

```
python3 scripts/benchmark_opensource_models.py \
    --config scripts/benchmark_models.example.json \
    --continue-on-failure \
    --timeout-seconds 7200
```

## D Environment Variables

The variables below are documented in `.env.example`. Exact final-run values should be archived with benchmark results.

**Table 9:** Documented environment variables.

| Variable        | Meaning                                                                       |
|-----------------|-------------------------------------------------------------------------------|
| LLM_BASE_URL    | OpenAI-compatible endpoint, default <code>http://localhost:8000/v1</code> .   |
| LLM_MODEL       | Chat model identifier, default <code>Qwen/Qwen2.5-Coder-14B-Instruct</code> . |
| LLM_API_KEY     | API key for the endpoint, default <code>EMPTY</code> .                        |
| LLM_TEMPERATURE | Generation temperature, default 0.                                            |

| Variable                           | Meaning                                                                          |
|------------------------------------|----------------------------------------------------------------------------------|
| LLM_MAX_TOKENS                     | Maximum generated tokens, default 4000.                                          |
| EMBEDDING_MODEL                    | Embedding model, default <code>sentence-transformers/all-MiniLM-L6-v2</code> .   |
| QDRANT_MODE                        | Qdrant mode: <code>local</code> , <code>server</code> , or <code>memory</code> . |
| QDRANT_PATH                        | Embedded Qdrant path, default <code>qdrant_storage</code> .                      |
| QDRANT_URL                         | Qdrant HTTP URL for server mode.                                                 |
| QDRANT_API_KEY                     | Optional Qdrant API key.                                                         |
| QDRANT_CONSTRAINT_COLLECTION       | Base name for constraint collections.                                            |
| QDRANT_TOOL_COLLECTION             | Base name for tool collections.                                                  |
| QDRANT_APP_*_CONSTRAINT_COLLECTION | Optional per-app constraint collection override.                                 |
| QDRANT_APP_*_TOOL_COLLECTION       | Optional per-app tool collection override.                                       |

## E Pending Items Checklist

### Pending item

- Start vLLM servers for all models in `scripts/benchmark_models.example.json`.
- Run the benchmark wrapper and archive the generated `benchmark_results/YYYYMMDD_HHMMSS/` directory.
- Fill the planned comparison table with measured values.
- Record GPU, CUDA, PyTorch, vLLM, model revision, and quantization settings.
- Pin dependencies or include a lockfile.
- Decide whether `qdrant_storage/` is a generated cache or a canonical delivered artifact.
- Add deterministic seeds to graph/data generation scripts where applicable.
- Add a single reproducibility entrypoint for the full artifact check.

## References

- [1] Association for Computing Machinery. Artifact review and badging version 1.1, 2020.
- [2] Jorge Echevarria de Uribarri. Meshagent repository, 2026. Inspected local commit 84218ab; full hash listed in the report appendix.
- [3] EURECOM. Newsroom: Download of our logo, 2026.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. Association for Computing Machinery, 2023.
- [5] Qdrant. Qdrant documentation, 2026.

- [6] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, 2019.
- [7] Aarne Rissanen. Investigating the reproducibility of llm-based network research – the case of netllm. Master’s thesis, University of Helsinki, 2026. Provided thesis PDF inspected for this report.
- [8] vLLM Project. vllm documentation, 2026.
- [9] Duo Wu, Xuan Wang, Yifan Qiao, Zhi Wang, Junchen Jiang, Shuguang Cui, and Fangxin Wang. Netllm: Adapting large language models for networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 661–678. Association for Computing Machinery, 2024.